Report No. 4B: UML Use Case Diagram + Extended Use Cases

Course Code: CSE 314
Course Title: Object Oriented Analysis and Design Laboratory

Submitted by

Tanvir Alam Shihab (334)

Murad Hossain Rohan (338)

K.M. Taskif Bin Parves (343)

Naosin Tabassum Lineya (364)

Marufa (365)

Submitted to

Dr. Md. Musfique Anwar, Professor

Sarnali Basak, Associate Professor

Computer Science and Engineering

Jahangirnagar University

Dhaka, Bangladesh

February 20, 2026

Contents

Abstract **iii**

1 OBJECTIVE **1**

2 EQUIPMENT USED **2**

3 UML Use Case Diagram **3**

3.1 Extended Use Cases: Sign-Up (2FA, Vendor Certificate, TIN, Shop Deletation),Security(Database) 4

3.2 Extended Use Cases: Offer and Theme Customization 8

3.3 Extended Use Cases: Payment & Delivery Management 11

3.4 Extended Use Cases: Notifications & Reviews Management 17

3.5 Extended Use Cases: Inventory & Analytics Management 24

4 INDIVIDUAL CONTRIBUTIONS **30**

5 DISCUSSION AND PROOFS **30**

5.1 Trello Activity Proof 30

5.2 Discord Communication Proof 31

5.3 Notification Activity Proof 32

6 CONCLUSIONS **33**

Abstract

This lab report presents the use case modeling for a Multi-Tenant E-Commerce System (M-TECS) , a cloud-based SaaS platform designed to assist small business owners who face challenges in acquiring customers through social media. The system provides an integrated solution for vendors to create customized online stores, manage products and offers, and process customer orders, while enabling customers to browse, purchase products, and track orders. The report includes two UML use case diagrams: the first illustrates the core system actors (Admin, Vendor, Customer) and their high-level interactions; the second provides a detailed breakdown of vendor-specific functionalities related to offers and store customization. These diagrams were created using Lucidchart to visually represent system requirements and actor-system relationships. The objectives, methodology, results, and conclusions of this modeling exercise are discussed, highlighting how use case diagrams serve as a foundational tool for system analysis and design in the OOAD lab context.

1. OBJECTIVE

The primary objectives of this lab exercise are as follows:

1. **To Identify System Actors and Use Cases:** Determine the primary actors interacting with the Multi-Tenant E-Commerce System (Admin, Vendor, Customer) and the key functionalities (use cases) each actor performs.

2. **To Model High-Level System Interactions:** Create a comprehensive UML use case diagram that illustrates the overall scope of the system and the relationships between actors and their core activities (registration, store management, ordering, etc.).

3. **To Elaborate Vendor-Specific Functionalities:** Develop a detailed use case diagram focusing specifically on the vendor's capabilities for creating offers and customizing their storefront, including complex relationships like `<<include>>` and generalizations.

4. **To Demonstrate UML Modeling Proficiency:** Apply object-oriented analysis and design principles using industry-standard modeling tools (Lucidchart) to visually document system requirements.

5. **To Establish a Foundation for Detailed Design:** Provide clear, visual documentation that bridges the gap between high-level project scope (as defined in the SRS) and detailed functional requirements for developers and stakeholders.

2. EQUIPMENT USED

The following tools and resources were used to complete this lab:

- **Modeling Tool:** Lucidchart (Web-based diagramming application)
- **Methodology:** Unified Modeling Language (UML) 2.x standards for Use Case Diagrams
- **System:** Windows/Linux PC with internet browser access
- **Reference Documents:** Software Requirements Specification (SRS) for Multi-Tenant E-Commerce System (Draft)

3. UML Use Case Diagram

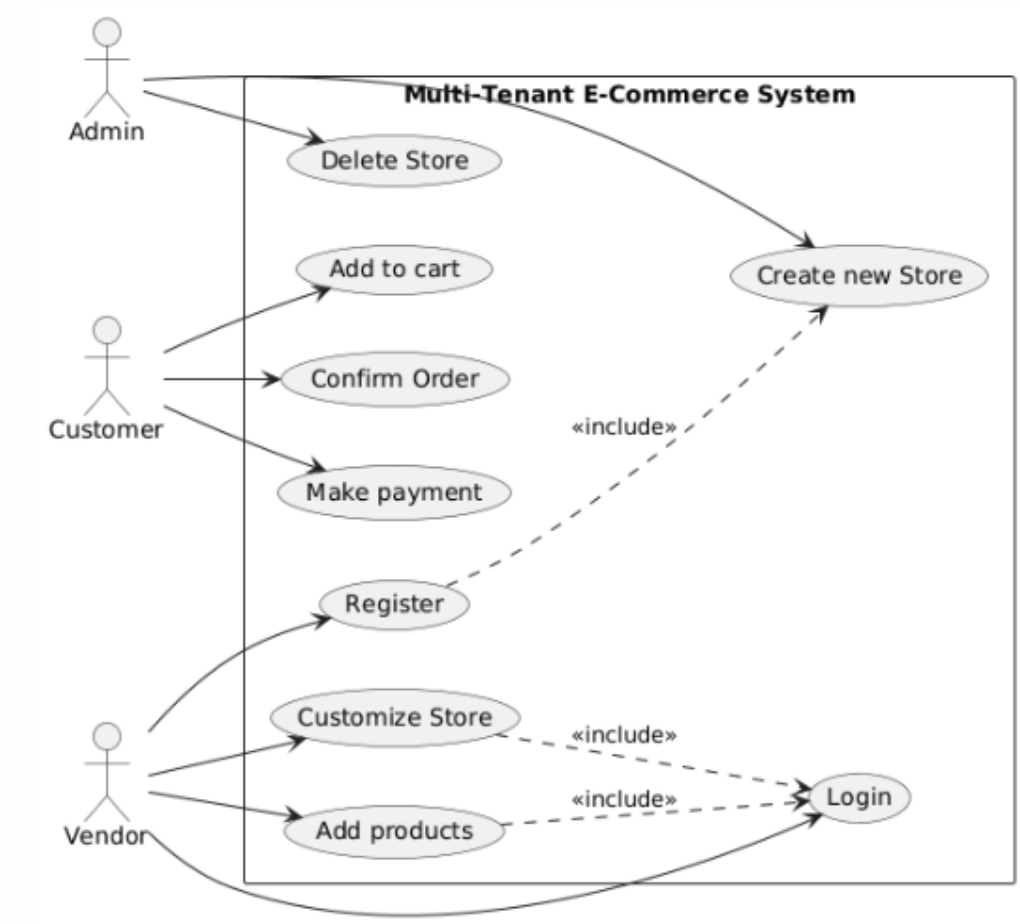

Figure 3.1: Multi-Tenant E-Commerce System Project UML Use case diagram

Description

This use case diagram provides a bird's-eye view of the entire Multi-Tenant E-Commerce System, illustrating the primary actors and their core interactions with the system. The diagram identifies three key actors: **Admin**, **Vendor**, and **Customer**.

- **Admin:** The system administrator is responsible for overarching governance. The diagram shows the Admin actor connected to two key use cases: *Delete*

Store (to remove inappropriate or inactive vendor stores) and *Login* (to access the admin panel). This actor represents the platform owner's operational control.

- **Vendor:** The small business owner (tenant) is the central user of the system. The diagram depicts the Vendor's core responsibilities, which include: *Register* (to create their tenant account), *Login* (to access their dashboard), *Add Products* (to populate their inventory), and *Customize Store* (to personalize their storefront's look and feel).
- **Customer:** The end-user who purchases products. The diagram shows the Customer's essential shopping journey: *Register* (to create an account), *Login* (to access their profile), *Add to Cart* (to select items), *Confirm Order* (to review and place the order), and *Make Payment* (to complete the transaction).

This high-level diagram effectively communicates the system's boundaries and the distinct roles of each user type, serving as an excellent starting point for stakeholder discussions and scope validation.

Extended Use Cases

3.1 Extended Use Cases: Sign-Up (2FA, Vendor Certificate, TIN, Shop Deletation),Security(Database)

Use Case 1: Vendor Registration & Subscription (FR-01)

Use Case Name: Vendor Onboarding and Subscription Enforcement

Primary Actor: Prospective Vendor

Secondary Actor: Payment Gateway (e.g., bKash/Nagad)

Goal: To legally verify a new business, collect the subscription fee, and create a new tenant account.

Preconditions:

1. The user possesses digital copies of their legal business documents.

Main Success Scenario:

1. The Prospective Vendor navigates to the registration page and inputs mandatory fields: Full Name, Business Email, Phone Number, Physical Address, Shop Type (Category), and a strong Password.
2. `<<include>>` The system provides an interface for the applicant to upload digital copies of mandatory legal documents: E-TIN Certificate, Trade License/Certificate, and E-Return Submission Proof.
3. The system validates the input format and enables the submission button.
4. Upon successful data entry and document upload, the system redirects the user to a payment gateway.
5. The vendor successfully completes the subscription payment.
6. The system finalizes the account creation process and redirects the user to the ”Registration Successful” page.

Extensions (Alternative Flows):

2a. **Missing Fields:** If any mandatory fields or documents are missing, the ”Sign Up” button is disabled.

4a. `<<extend>>` **Payment Failure:** If the payment fails, the user is returned to the payment screen with an error message. The account remains in a ”Pending” state until the subscription is active.

Post Condition:

1. **Successful registration:** New tenant account is created with active subscription; vendor can access their store dashboard.
2. **Pending registration:** Account created but marked as ”Pending” until payment is successfully completed.
3. **Failed registration:** No account created; vendor remains on registration page to correct errors.

Use Case 2: Secure Login with 2FA (FR-01)

Use Case Name: Secure Vendor Authentication

Primary Actor: Registered Vendor

Goal: To securely authenticate the vendor and prevent unauthorized access to financial and order history data.

Preconditions:

1. The vendor has an active, fully registered account.

Main Success Scenario:

1. The Vendor enters their registered username and password on the login page.
2. The system verifies the credentials.
3. The system displays a ”Verify OTP” screen.
4. `<<include>>` The system sends a One-Time Password (OTP) to the vendor’s registered email or phone number within 10 seconds.
5. The Vendor enters the received OTP.
6. The system verifies that the entered OTP matches the generated code.
7. The system grants access to the vendor’s dashboard.

Extensions (Alternative Flows):

2a. **Invalid Credentials:** If username or password is incorrect, system displays error message and remains on login page.

4a. **OTP Delivery Failure:** If OTP cannot be delivered, system provides option to resend OTP or contact support.

6a. **Invalid OTP:** Access is denied if the entered OTP does not match the generated code. System displays error message and allows OTP re-entry up to 3 attempts.

6b. **OTP Expired:** If OTP is not entered within the validity period (e.g., 5 minutes), system displays ”OTP Expired” and prompts to request a new OTP.

Post Condition:

1. **Successful authentication:** Vendor is granted access to their dashboard with a secure session.
2. **Failed authentication:** Vendor remains on login/OTP screen; access is denied; failed attempts are logged for security monitoring.

Use Case 3: Database Provisioning & Data Routing (FR-02)

Use Case Name: Automated Database Provisioning and Data Isolation

Primary Actor: System (Super Admin)

Goal: To automatically isolate tenant data and ensure queries are securely routed only to the correct vendor's database.

Preconditions:

1. A new vendor has just successfully completed registration and their account is approved.

Main Success Scenario:

1. The System detects a newly approved vendor registration.
2. `<<include>>` The System stores the vendor's global information, including Vendor Login Credentials, Subscription Status, and Tenant Metadata (Name, Shop URL), in the Central Database (Catalog DB).
3. The System automatically provisions a distinct, isolated database dedicated solely to that vendor.
4. The System initializes the new database with standard tables (Products, Orders, Customers) containing zero data.
5. When the vendor makes a subsequent request (e.g., viewing "All Orders"), the System middleware identifies the incoming request's origin via the Tenant ID.
6. The System dynamically routes the query to the correct isolated database, returning only their private business data.

Extensions (Alternative Flows):

2a. **Metadata Storage Failure:** If storing vendor information in Central Database fails, the system logs the error and notifies the Super Admin. Vendor account creation is rolled back.

3a. **Database Provisioning Failure:** If automatic database creation fails, the system retries up to 3 times. After repeated failures, the system flags the vendor account for manual intervention and notifies the Super Admin.

5a. **Attempted Data Breach:** If a query attempts to access another vendor's data (Cross-tenant SQL query), the system strictly prohibits it at the application level to ensure Vendor A cannot access Vendor B's data. The attempted breach is logged for security audit.

5b. **Invalid Tenant ID:** If the Tenant ID is missing or invalid, the system rejects the request and returns an authentication error.

Post Condition:

1. **Successful provisioning:** New vendor has a dedicated, isolated database; all subsequent queries are correctly routed to the appropriate tenant database; data isolation is maintained.
2. **Failed provisioning:** Vendor account is flagged; Super Admin is notified; manual intervention may be required to complete the provisioning process.
3. **Security violation:** Any cross-tenant access attempts are blocked and logged; data integrity and isolation are preserved.

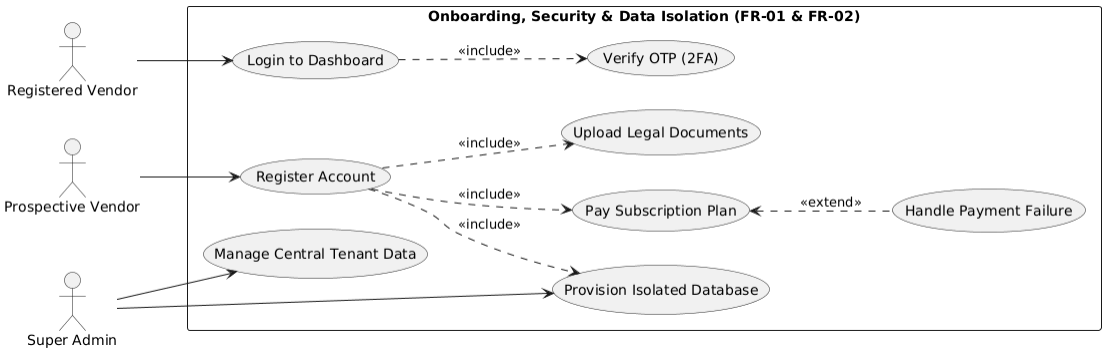

Figure 3.2: Onboarding, security and data isolation

3.2 Extended Use Cases: Offer and Theme Customization

Use Case Scenario 3: Create Percentage Discount with Deadline

Use Case Name: Create Percentage Discount with Deadline

Actor: Vendor

Scenario / Description:

1. Vendor logs into their store dashboard.
2. Vendor navigates to ”Offers & Promotions” section.
3. Vendor selects ”Create New Offer” and chooses ”Percentage Discount.”
4. Vendor enters discount details:
   (a) Discount percentage (e.g., 10%, 20%, 50%)
   (b) Products or categories to apply discount to
   (c) Minimum purchase amount (if applicable)
5. System enforces `<<include>>` relationship: Vendor must set an offer deadline.
6. Vendor selects offer start date and end date (deadline).
7. System validates that end date is after start date and not in the past.
8. Vendor optionally sets a coupon limit (maximum number of redemptions).
9. Vendor reviews and confirms the discount offer details.
10. System saves the discount offer and activates it according to the schedule.
11. System displays the new offer in the vendor’s active promotions list.

Exception:

1. Deadline (end date) is not set (violates mandatory `<<include>>` requirement).
2. End date is earlier than start date.
3. Start date or end date is in the past.
4. Discount percentage is invalid (e.g., negative, zero, or ¿100%).
5. No products or categories selected for the discount.
6. Coupon limit is set to an invalid value (negative or zero when required).
7. Database connection failure during offer creation.

Precondition:

1. Vendor is logged into the system with valid session.
2. Vendor has products listed in their inventory.
3. System database is operational.

Post Condition:

1. **Successful discount creation:** New percentage discount offer is created and saved in the system; offer is scheduled according to start/end dates; customers can now avail the discount on eligible products; offer appears in vendor's promotions list.
2. **Unsuccessful discount creation:** Discount offer is not saved; system displays appropriate error messages (e.g., "Deadline must be set," "End date must be after start date"); vendor remains on offer creation page to correct errors.
3. **Exception:** Transaction is rolled back; system displays error message; vendor remains on offer creation page if possible.

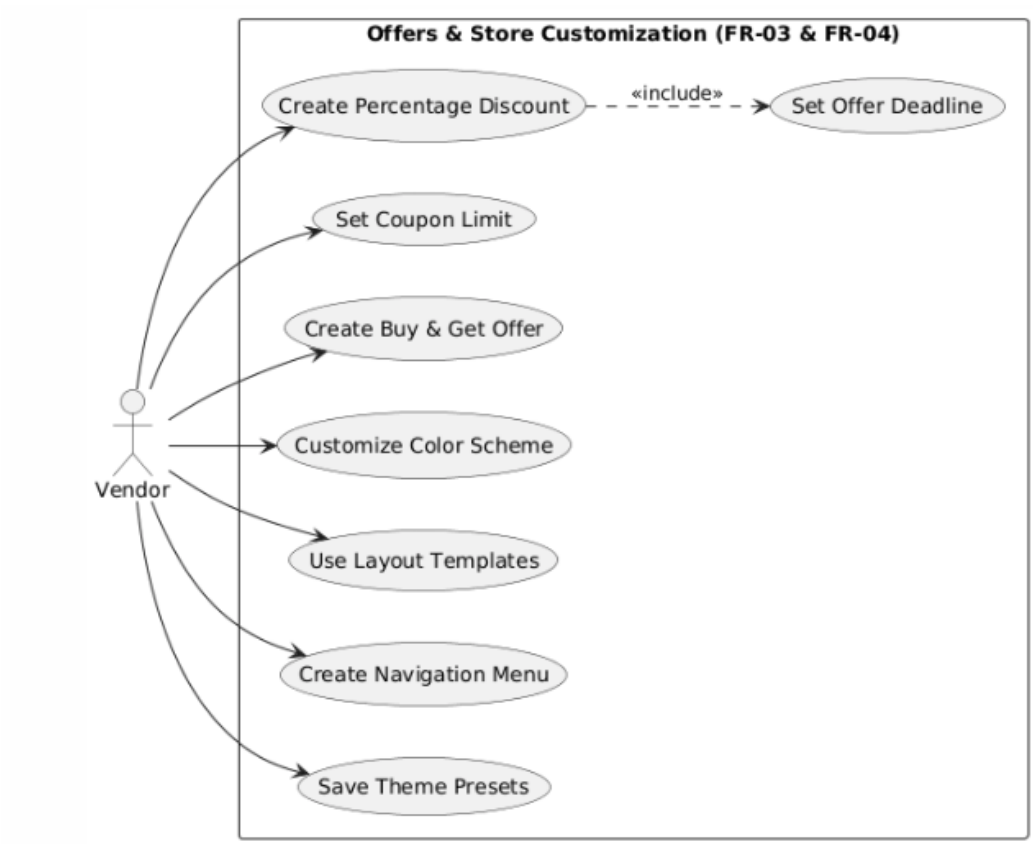

Figure 3.3: Vendor specific customization and discount offers' UML Use case diagram

3.3 Extended Use Cases: Payment & Delivery Management

Use Case 1: Multi-Method Payment Processing (FR-05)

Use Case Name: Process Order Payment

Primary Actor: Customer

Secondary Actor: System, Payment Gateway (External)

Goal: To allow customers to pay for orders using multiple online payment gateways or Cash on Delivery (COD).

Preconditions:

1. The Customer has items in the shopping cart.
2. The Customer is logged into the system.

Main Success Scenario:

1. The Customer proceeds to checkout.
2. The Customer selects a preferred payment method (online or COD).
3. If an online payment is selected, the system redirects the Customer to the chosen payment gateway.
4. The Customer completes payment through the gateway.
5. The system verifies the payment status.
6. The system updates the order status as ”Paid” or ”Failed”.
7. If COD is selected, the system marks payment status as ”Pending (COD)”.
8. The system confirms the order and stores payment details.

Extensions (Alternative Flows):

3a. **Payment Failure:** If the online payment fails, the system cancels order confirmation and displays an error message.

3b. **Invalid Payment Method:** If the selected payment method is unavailable, the system prompts the Customer to choose a different method.

4a. `<<extend>>` **Payment Gateway Timeout:** If the payment gateway does not respond within the specified time, the system cancels the transaction and notifies the customer to try again.

Post Condition:

1. **Successful payment:** Order is confirmed with status ”Paid” (online) or ”Pending (COD)”; payment details are stored.
2. **Failed payment:** Order is not confirmed; customer remains on checkout page with error message.

Use Case 2: Payment Method Selection (FR-05)

Use Case Name: Choose Payment Method

Primary Actor: Customer

Secondary Actor: System

Goal: To allow the Customer to select a preferred payment method during checkout.

Preconditions:

1. The Customer has items in the shopping cart.
2. The Customer is logged into the system.

Main Success Scenario:

1. The Customer navigates to the checkout page.
2. The system displays all available payment options (bKash, Nagad, Visa, MasterCard, COD).
3. The Customer selects a payment method.
4. The system shows the selected method in the order summary.
5. The Customer submits the order.

6. The system confirms order creation based on the chosen payment method.

Extensions (Alternative Flows):

2a. **No Payment Methods Available:** If no payment methods are configured or available, the system displays an error message and prevents order submission.

3a. **Method Unavailable:** If the selected payment method is temporarily unavailable, the system prompts the Customer to choose an alternative method.

5a. **Order Submission Without Selection:** If the Customer attempts to submit the order without selecting a payment method, the system highlights the required field and prevents submission.

Post Condition:

1. **Successful selection:** Payment method is recorded with the order; order processing continues based on the selected method.
2. **Unsuccessful selection:** Order is not submitted; customer remains on checkout page.

Use Case 3: Online Payment Verification (FR-05)

Use Case Name: Verify Online Payment

Primary Actor: Customer

Secondary Actor: System, Payment Gateway (External)

Goal: To ensure the Customer's online payment is successfully completed before confirming the order.

Preconditions:

1. The Customer has selected an online payment method.
2. The Customer has proceeded to the payment gateway.

Main Success Scenario:

1. The system redirects the Customer to the payment gateway.
2. The Customer completes payment through the gateway interface.
3. The system receives payment confirmation from the gateway (callback/webhook).
4. If payment succeeds, the system marks the order as ”Paid”.
5. If payment fails, the system shows an error and does not confirm the order.
6. The system redirects the Customer back to the order confirmation or failure page.

Extensions:

3a. **Payment Timeout:** If the payment process times out, the system cancels the transaction and notifies the Customer to try again.

3b. **Duplicate Payment:** If the same payment is processed twice, the system detects duplication and initiates refund for the duplicate transaction.

3c. **Invalid Callback:** If the system receives an invalid or unverified callback from the gateway, it rejects the update and logs the incident for security review.

Post Condition:

1. **Payment successful:** Order status updated to ”Paid”; customer receives confirmation; inventory is updated.
2. **Payment failed:** Order remains in ”Pending Payment” or is cancelled; customer is prompted to retry payment.

Use Case 4: Product Delivery Tracking (FR-06)

Use Case Name: Track Order Delivery

Primary Actor: Customer

Secondary Actor: System, Third-Party Courier API (Pathao, RedX, etc.)

Goal: To provide real-time tracking of order delivery via third-party courier services.

Preconditions:

1. The Customer has placed an order.
2. The order has been dispatched via a third-party courier.
3. A valid tracking ID has been generated and associated with the order.

Main Success Scenario:

1. The Customer navigates to the order details page.
2. The system fetches delivery status from courier APIs (Pathao, RedX, etc.) using the tracking ID.
3. The system displays the current delivery status with timestamp (e.g., Picked Up, In Transit, Out for Delivery, Delivered).
4. The delivery status updates automatically as the shipment progresses.
5. Upon delivery completion, the system marks the order as ”Delivered” and optionally requests feedback.

Extensions:

2a. **API Unavailable:** If the courier API is temporarily unavailable, the system displays the last known status and retries connection periodically.

2b. **Invalid Tracking ID:** If the tracking ID is invalid or not found, the system displays a message asking the customer to contact support.

3a. **Delivery Exception:** If the courier reports an exception (e.g., address not found, recipient unavailable), the system displays the exception details and suggests next steps.

Post Condition:

1. **Successful tracking:** Customer can view real-time delivery status; system maintains synchronization with courier updates.
2. **Delivery completed:** Order status updated to ”Delivered”; customer may be prompted to rate their experience.

Use Case 5: Vendor Delivery Monitoring (FR-06)

Use Case Name: Monitor Shipment Status

Primary Actor: Vendor

Secondary Actor: System, Third-Party Courier API (Pathao, RedX, etc.)

Goal: To allow Vendors to monitor delivery status of customer orders in real-time.

Preconditions:

1. The Vendor is logged into the system.
2. Orders have been confirmed and dispatched with valid tracking IDs.

Main Success Scenario:

1. The Vendor navigates to their dashboard or ”Orders” section.
2. The system displays a list of all orders with their current status.
3. For dispatched orders, the system retrieves courier-provided tracking information for each order via API integration.
4. The Vendor views delivery progress for each order (Picked Up, In Transit, Delivered, Returned).
5. The system automatically updates statuses as the courier updates the shipment.
6. The Vendor can filter orders by delivery status to identify pending, delivered, or returned shipments.

Extensions:

3a. **Bulk Status Update:** The system periodically (e.g., every 30 minutes) batches API calls to update all active shipments efficiently.

3b. **Delivery Exception Alert:** If a shipment encounters an exception (e.g., return to sender), the system highlights the order and optionally sends a notification to the Vendor.

4a. **Manual Status Override:** In case of API failure, the Vendor may have the ability to manually update delivery status (with appropriate audit logging).

5a. **Export Delivery Report:** The Vendor can export delivery data for reconciliation and accounting purposes.

Post Condition:

1. **Successful monitoring:** Vendor has up-to-date visibility of all shipment statuses; can take action on exceptions.
2. **Delivery completed:** Orders marked as delivered are moved to completed orders; revenue can be reconciled.
3. **Exception handling:** Returned or undelivered orders are flagged for Vendor action (e.g., customer contact, refund processing).

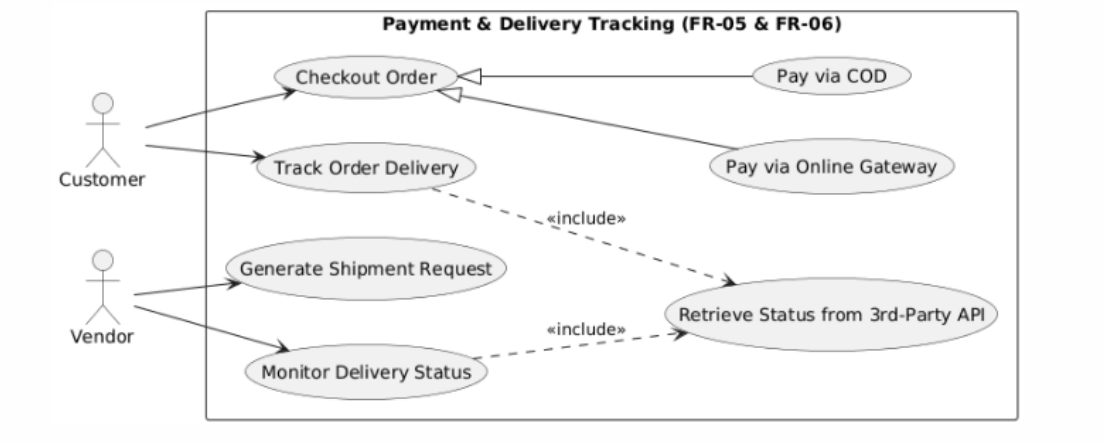

Figure 3.4: Payment & Delivery Management

3.4 Extended Use Cases: Notifications & Reviews Management

Use Case 1: Promotional Notification Management (FR-09)

Use Case Name: Send Promotional Notification

Primary Actor: Vendor

Secondary Actor: System, Notification Service (Email/SMS/Push)

Goal: To notify customers about promotions, offers, or announcements to increase engagement and sales.

Preconditions:

1. The Vendor has an active account.
2. The Vendor is logged into the dashboard.
3. The system has customer contact information (email/phone) for selected audience.

Main Success Scenario:

1. The Vendor navigates to the promotional notifications section from the dashboard.
2. The Vendor selects the option to create a new promotional notification.
3. The Vendor enters the notification content (offer details, message text).
4. `<<include>>` The system prompts the Vendor to select the target audience.
5. The Vendor selects the desired customer group (all customers or a specific segment based on purchase history, location, etc.).
6. The Vendor submits the promotional notification.
7. The system sends the notification to the selected customers via configured channels (email, SMS, in-app notification).
8. The system stores the notification details in the notification history for future reference.

Extensions (Alternative Flows):

4a. **No Customer Segments Available:** If no customer segments are defined, the system defaults to ”All Customers” and notifies the Vendor.

5a. **Empty Audience Selection:** If the Vendor selects a segment with zero customers, the system warns and prompts to select a different audience or cancel.

7a. **Notification Delivery Failure:** If some notifications fail to send (e.g., invalid email/phone), the system logs failures and continues sending to remaining customers.

7b. `<<extend>>` **Bulk Sending Rate Limit:** If sending to a large audience, the system implements rate limiting to avoid being flagged as spam.

Post Condition:

1. **Successful sending:** Promotional notifications are delivered to target customers; notification is recorded in history with timestamp and audience size.
2. **Partial success:** Some notifications failed but were logged; Vendor can view delivery report in history.
3. **Failure:** No notifications sent; Vendor remains on creation page with error message.

Use Case 2: View Notification History (FR-09)

Use Case Name: View Notification History

Primary Actor: Vendor

Secondary Actor: System

Goal: To review previously sent promotional notifications for tracking and analysis.

Preconditions:

1. The Vendor is logged into the system.
2. Previous promotional notifications have been sent.

Main Success Scenario:

1. The Vendor navigates to the notification history section from the dashboard.
2. The system retrieves all previously sent promotional notifications from the database.
3. The system displays the notification list with the following details for each:
   (a) Date and time sent
   (b) Notification title/message preview
   (c) Target audience (e.g., ”All Customers” or segment name)
   (d) Delivery status (Sent, Failed, Partial)
   (e) Open/click rate (if tracking is enabled)

4. The Vendor can filter or search through the history by date range or message content.
5. The Vendor selects a specific notification to view full details and delivery report.

Extensions (Alternative Flows):

2a. **No History Available:** If no notifications have been sent yet, the system displays a message ”No notification history found” with a link to create a new notification.

3a. **Analytics Unavailable:** If engagement tracking (opens/clicks) is not configured, the system displays ”Tracking not available” for those fields.

5a. **Export History:** The Vendor may have the option to export notification history as CSV/PDF for reporting purposes.

Post Condition:

1. **Successful view:** Vendor can review all past promotional communications; data can be used to analyze campaign effectiveness.
2. **No data:** Vendor is informed that no notifications have been sent yet.

Use Case 3: Customer Review & Rating Submission (FR-10)

Use Case Name: Submit Product Review and Rating

Primary Actor: Customer

Secondary Actor: System

Goal: To allow customers to submit feedback for purchased products, helping other customers make informed decisions.

Preconditions:

1. The Customer has completed a purchase (order status: Delivered).
2. The Customer is logged into the system.
3. The product is still available in the system catalog.

Main Success Scenario:

1. The Customer navigates to the order history page.
2. The Customer selects a purchased product from a delivered order to review.
3. The Customer enters a rating (1–5 stars) and written feedback (optional but encouraged).
4. `<<include>>` The system verifies that the product was actually purchased by the Customer (checks order history).
5. The Customer submits the review.
6. The system validates the input (rating range, content length).
7. The system saves the review in the database and associates it with the product and customer.
8. The system recalculates and updates the product's average rating based on all reviews.
9. The review becomes visible on the product details page for all customers to see.

Extensions (Alternative Flows):

2a. **Already Reviewed:** If the Customer has already reviewed this product, the system displays "You have already reviewed this product" and allows editing instead of new submission.

4a. **Purchase Not Verified:** If the product was not purchased by this Customer, the system denies the review submission and displays "You can only review products you have purchased."

6a. **Invalid Rating:** If the rating is outside the allowed range (e.g., 0 or 6), the system displays an error message "Rating must be between 1 and 5 stars."

6b. **Inappropriate Content:** If the review contains prohibited words/phrases, the system flags it for moderation or rejects with an appropriate message.

7a. **Database Error:** If saving fails, the system displays an error and prompts the Customer to try again later.

Post Condition:

1. **Successful submission:** New review is live on the product page; product's average rating is updated; other customers can see the feedback.
2. **Failed submission:** Review is not saved; Customer receives error message and can correct and resubmit.
3. **Flagged review:** Review is saved but marked for moderation; not visible publicly until approved.

Use Case 4: Flag Inappropriate Review (FR-10)

Use Case Name: Flag Inappropriate Review

Primary Actor: Vendor

Secondary Actor: System, Administrator (for final action)

Goal: To report inappropriate, fraudulent, or misleading customer reviews for moderation and potential removal.

Preconditions:

1. The Vendor is logged into the system.
2. A customer review exists for the Vendor's product.
3. The Vendor has identified a review that violates platform guidelines.

Main Success Scenario:

1. The Vendor navigates to the product reviews section for one of their products.
2. The Vendor browses through customer reviews and identifies an inappropriate review.
3. The Vendor selects the review and clicks the "Flag" or "Report" option.
4. The system prompts the Vendor to select a reason for flagging (e.g., Spam, False Information, Abusive Language, Off-topic).
5. The Vendor optionally adds additional comments or context.
6. The Vendor submits the flag request.

7. The system marks the review as ”Flagged” in the database and changes its status to ”Under Review.”
8. The system sends a notification to the Administrator for manual moderation.
9. The review is temporarily hidden from public view or remains visible pending review (based on platform policy).

Extensions (Alternative Flows):

4a. **No Reason Selected:** If the Vendor attempts to flag without selecting a reason, the system prompts ”Please select a reason for flagging this review.”

7a. **Auto-Moderation:** Based on platform configuration, the system may automatically hide reviews flagged for certain reasons (e.g., hate speech) without admin intervention.

8a. **Duplicate Flag:** If the same review has already been flagged by another Vendor/Customer, the system increments the flag count and notifies ”This review has already been reported.”

9a. **Administrator Decision:** After review, the Administrator may:

(a) Remove the review entirely if found inappropriate

(b) Restore the review if flagged in error

(c) Warn the Customer who posted the review

Post Condition:

1. **Successful flagging:** Review is marked ”Under Review”; Administrator is notified; temporary action may be taken based on policy.
2. **Review removed:** If Administrator confirms violation, review is permanently removed from product page.
3. **Review restored:** If flagged in error, review is reinstated and flag is removed.
4. **Vendor notified:** Vendor receives notification of the final decision on their flag request.

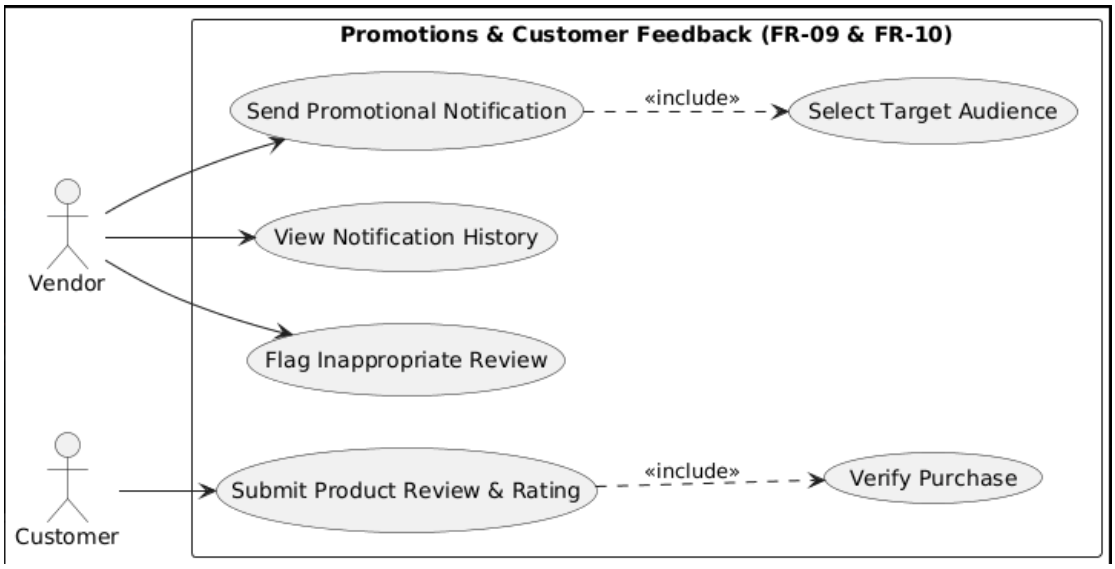

Figure 3.5: Promotions and customer feedback

3.5 Extended Use Cases: Inventory & Analytics Management

Use Case 1: Manage Inventory (FR-07)

Use Case Name: Manage Inventory

Primary Actor: Vendor

Secondary Actor: System

Goal: To provide vendors full control over product lifecycle and maintain accurate stock levels.

Preconditions:

- The Vendor has an active account and is logged into the system.

Main Success Scenario:

1. The Vendor navigates to the Inventory Management section of the dashboard.
2. The Vendor selects the option to add a new product.
3. The Vendor inputs the product name, price, SKU, and initial quantity.

4. The system validates the input and stores the product in the central database.
5. The Vendor sets a ”Low Stock” threshold for the product.
6. The system confirms the product is live and synchronized with the digital storefront.

Extensions (Alternative Flows):

3a. **Manual Stock Update:** The Vendor edits an existing product’s quantity.

- **Validation:** The system implements rules to prevent stock quantities from falling below zero.

3b. **Remove Product:** The Vendor removes a product from the system, and the storefront reflects this change instantly.

4a. **Invalid Input:** If the vendor enters invalid data (negative price, duplicate SKU, etc.), the system displays an error message and prevents submission.

5a. **Threshold Validation:** If the low stock threshold is set higher than the initial quantity, the system warns the vendor but allows the setting.

Post Condition:

- **Successful addition:** New product is live in the storefront with accurate initial stock.
- **Successful update:** Existing product stock levels are updated and reflected immediately.
- **Successful removal:** Product is no longer visible in the storefront.
- **Failed operation:** No changes are made; vendor receives appropriate error message.

Use Case 2: View Analytics Dashboard (FR-08)

Use Case Name: View Analytics Dashboard

Primary Actor: Vendor

Secondary Actor: System

Goal: To translate raw sales data into actionable insights through visual charts.

Preconditions:

- The Vendor is logged into the system and has existing sales data.

Main Success Scenario:

1. The Vendor navigates to the Analytical Board from the main menu.
2. The system retrieves real-time sales data from the database.
3. The system calculates and displays Key Performance Indicators (KPIs) including total sales, revenue, and order counts.
4. The system renders a ranked list of best-selling items.
5. The system displays revenue trends using bar charts and product distribution using pie charts.

Extensions (Alternative Flows):

2a. **No Data Available:** If the vendor has no sales data yet, the system displays a message ”No sales data available” with suggestions to start promoting products.

3a. **Data Calculation Delay:** If data volume is large, the system shows a loading indicator while calculating KPIs.

5a. **Chart Rendering Error:** If charts fail to render, the system displays data in tabular format as fallback.

Post Condition:

- **Successful view:** Vendor can see comprehensive sales analytics with visual representations.
- **No data:** Vendor is informed that no sales data exists yet.

Use Case 3: Filter Data by Date (FR-08 Extension)

Use Case Name: Filter Data by Date

Primary Actor: Vendor

Secondary Actor: System

Goal: To refine dashboard visualizations and metrics by a user-defined timeframe.

Relationship: `<<extend>>` to View Analytics Dashboard

Preconditions:

- The Vendor is viewing the Analytics Dashboard.
- Sales data exists for the selected date range.

Main Success Scenario:

1. While viewing the Analytics Dashboard, the Vendor selects the date-picker utility.
2. The Vendor defines a specific start and end date.
3. The system filters all charts (Bar/Pie) and KPI lists based on the selected timeframe.
4. The system updates the visuals instantly without requiring a full page reload.

Extensions (Alternative Flows):

2a. **Invalid Date Range:** If the Vendor selects an end date earlier than the start date, the system displays an error and resets the selection.

2b. **Future Dates:** If the Vendor selects future dates, the system shows data up to the current date and indicates that future dates have no data.

3a. **No Data in Range:** If no sales exist for the selected date range, the system displays ”No data available for selected period” and shows empty charts.

4a. **Reset Filter:** The Vendor can clear the date filter to return to the default view (e.g., last 30 days or all time).

Post Condition:

- **Successful filter:** All dashboard elements reflect data only from the selected date range.
- **Reset:** Dashboard returns to default timeframe view.
- **No data:** Vendor is informed that no data exists for the selected period.

Use Case 4: Automated Stock Synchronization (FR-07)

Use Case Name: Update Stock Level

Primary Actor: Customer

Secondary Actor: System, Vendor (indirectly notified)

Goal: To ensure real-time synchronization between customer purchases and physical availability.

Relationship: `<<include>>` in Confirm Order

Preconditions:

- The Customer has items in their cart.
- The Customer completes the checkout process.

Main Success Scenario:

1. The Customer completes a checkout and confirms an order.
2. The system automatically decrements the stock levels for the purchased items in the database.
3. The system updates the Vendor's dashboard in real-time.
4. **Low Stock Trigger:** If a product's quantity falls below the pre-set threshold, the system triggers an automated dashboard notification for the Vendor.

Extensions (Alternative Flows):

2a. **Insufficient Stock:** If the ordered quantity exceeds available stock, the system:
[label*=5.]

1. Prevents order confirmation

2. Notifies the Customer about insufficient stock
3. Suggests reducing quantity or removing the item

2b. **Stock Update Failure:** If database update fails, the system rolls back the entire order transaction and notifies both Customer and Vendor.

3a. **Concurrent Orders:** If multiple customers order the same product simultaneously, the system implements row-level locking to prevent overselling.

4a. **Multiple Low Stock Alerts:** The system consolidates low stock notifications to avoid spamming the Vendor (e.g., one notification per product per hour).

4b. **Critical Stock Alert:** If stock reaches zero, the system automatically hides the product from the storefront and sends an urgent notification to the Vendor.

Post Condition:

- **Successful synchronization:** Stock levels accurately reflect all confirmed orders; Vendor dashboard shows updated inventory; Customer sees accurate availability.
- **Low stock alert:** Vendor is notified to replenish inventory.
- **Out of stock:** Product is automatically removed from storefront until restocked.
- **Oversell prevented:** No order is confirmed that would exceed available inventory.

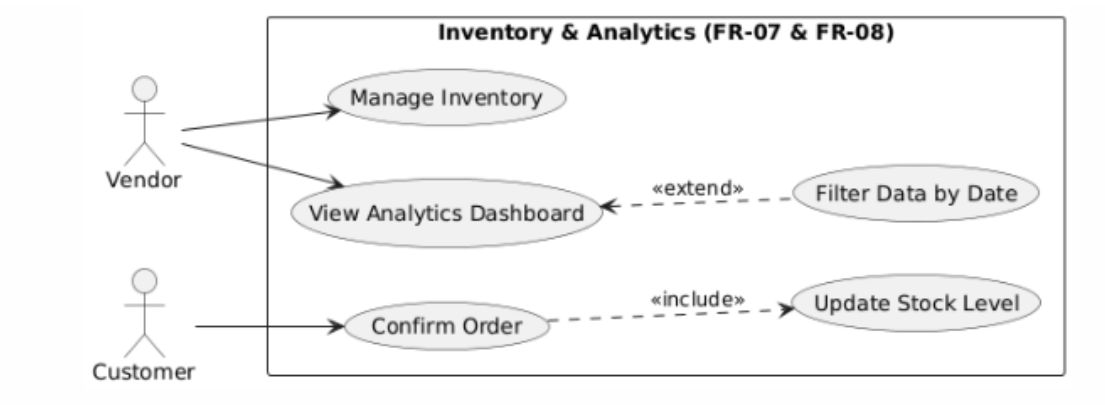

Figure 3.6: Inventory and analytics

4. INDIVIDUAL CONTRIBUTIONS

I was assigned Functional Requirements 7 and 8, titled Manage Inventory (FR-07) and View Analytics Dashboard (FR-08), respectively. Therefore, I worked on designing and implementing the inventory management module, which includes adding, updating, deleting, and tracking stock levels of products. Additionally, I developed the analytics dashboard feature to display key performance indicators, sales reports, and business insights through graphical representations. I also contributed to the extended UML use case diagram based on these functional requirements.

5. DISCUSSION AND PROOFS

This chapter presents discussion and visual proof of the activities performed during Lab Activity 4B – UML Use Case Diagram and Extended Use Cases documentation.

During this lab activity, the team designed the UML Use Case Diagram and developed detailed Extended Use Cases for the system. Trello was used for task management and progress tracking, Discord was used for communication and coordination, and system notifications were used to ensure timely updates and reminders.

These tools ensured proper organization, collaboration, and timely completion of the UML and documentation tasks.

5.1 Trello Activity Proof

Trello was used to organize and monitor tasks related to drawing the UML Use Case Diagram and writing the Extended Use Cases. Tasks were categorized into **To Do**,

Doing, and **Done** sections to clearly track progress.

Responsibilities such as diagram design, actor identification, use case description writing, and review were assigned to team members to maintain accountability and coordination.

The screenshots below demonstrate task planning and task completion during Lab Activity 4B.

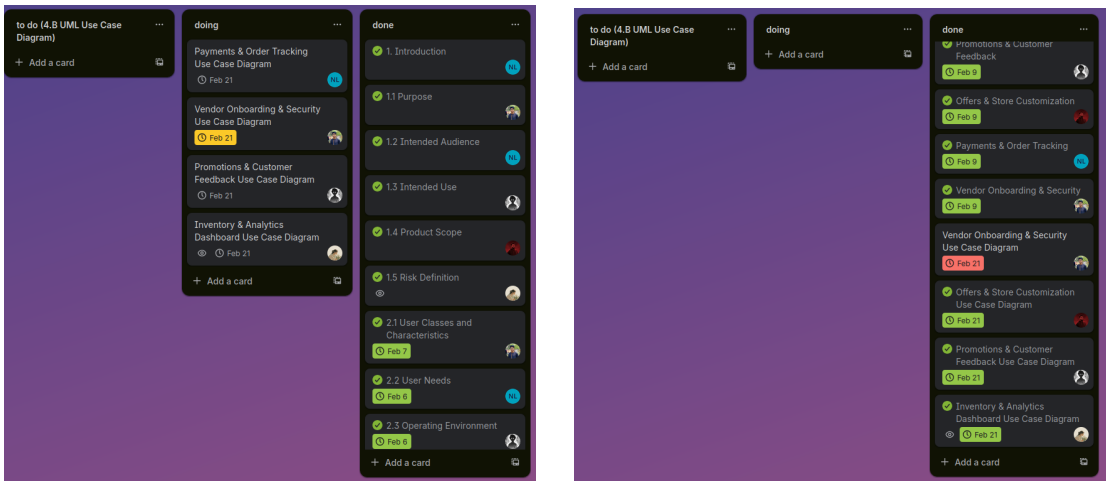

Figure 5.1: Trello board showing UML Use Case Diagram and Extended Use Case task management

5.2 Discord Communication Proof

Discord was used as the primary communication platform during Lab Activity 4B. Team members discussed system actors, clarified use case relationships, confirmed diagram structure, and coordinated documentation formatting.

It also supported file sharing of the UML diagrams and draft Extended Use Case descriptions for review and improvement.

The screenshots below show discussion, coordination, and submission-related communication.

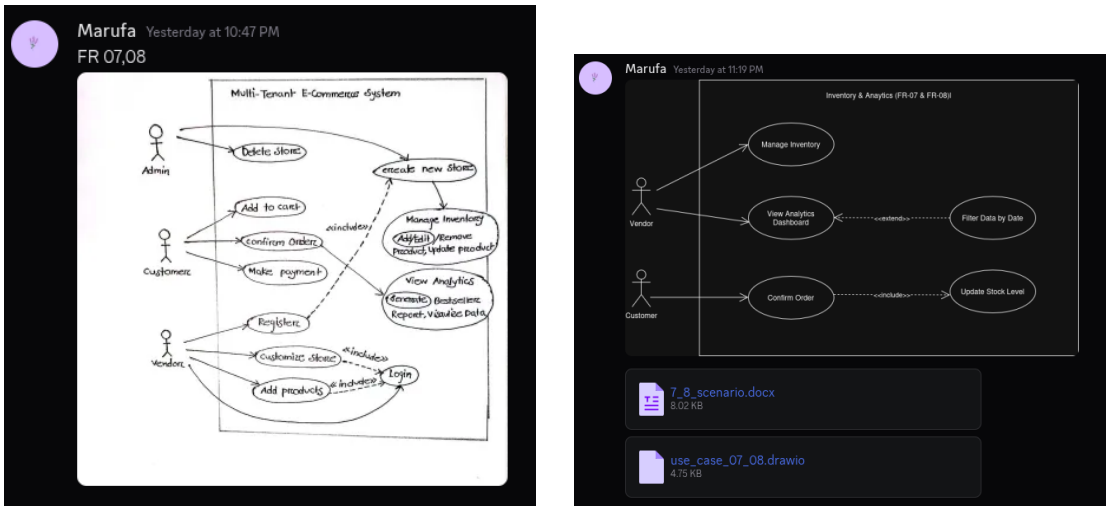

Figure 5.2: Discord communication showing discussion and coordination during UML development

5.3 Notification Activity Proof

Notifications played an important role in ensuring that deadlines were met and tasks were completed on time. Trello notifications were used to alert team members about task assignments, updates, and approaching deadlines. Discord notifications ensured immediate awareness of important discussions and file updates.

These notifications helped improve responsiveness and maintain consistent progress throughout the lab activity.

The screenshots below demonstrate notification alerts related to task updates and team communication.

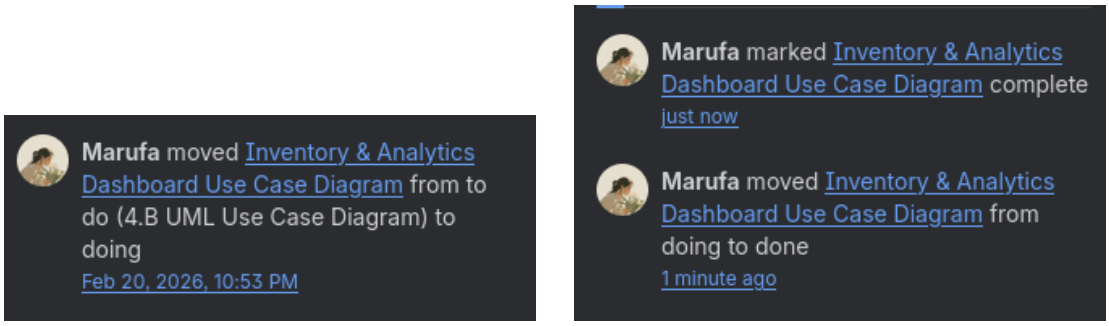

Figure 5.3: Notification alerts showing task updates and communication reminders

6. CONCLUSIONS

This lab exercise successfully demonstrated the application of UML use case diagrams to model the requirements of the Multi-Tenant E-Commerce System. By creating both a high-level system diagram and a detailed vendor-focused diagram, we achieved the stated objectives of identifying actors, clarifying system scope, and elaborating on key functionalities like offer management and store customization.

The use of Lucidchart proved effective for creating professional, clear, and shareable UML diagrams. The `<<include>>` relationship identified in the offers diagram highlights how use case modeling can capture critical business logic early in the development lifecycle, preventing ambiguities during the coding phase.